

Stored Procedure

Stored Procedure

- São funções escritas usando SQL e ficam armazenadas no servidor
- Podem receber parâmetros e executar funções complexas, retornando ou não as informações para o usuário

Vantagens-Stored Procedure

- Minimizam o tráfego da rede
- São mais rápidos, aproveitam a capacidade do servidor e da otimização do SGBD
- Facilitam a manutenção. Pode ser acessada por diversos programas, se houver alguma modificação a ser feita, basta fazê-la no BD

Limitações-Stored Procedure

- Passar qualquer informação variável para a stored procedure como parâmetros ou colocá-las em uma tabela que a stored procedure possa acessar.
- A linguagem de escrita de stored procedures pode ser muito limitada para operações mais complexas.

Sintaxe-Stored Procedures

```
CREATE PROCEDURE nome_da_procedure(parametros)
LANGUAGE plpgsql
AS $$
BEGIN
    bloco de comandos
END;
$$;
```

Sintaxe-Stored Procedures

CREATE PROCEDURE nome_da_procedure(parametros)

- **CREATE PROCEDURE:** Informa ao PostgreSQL que estás criando uma **procedure** (procedimento armazenado).
- **nome_da_procedure:** É o **nome** da procedure, que será usado posteriormente para chamá-la com **CALL nome_da_procedure(...)**.
- **(parametros):** Define os **parâmetros de entrada ou saída** da procedure. Cada parâmetro tem:
 - Uma direção (**IN**, **OUT**, **INOUT**)
 - Um nome (ex: **cliente_id**)
 - Um tipo de dado (ex: **INT**, **TEXT**, etc.)

Exemplo: (IN nome TEXT, IN idade INT)

Sintaxe-Stored Procedures

LANGUAGE plpgsql

- Define a **linguagem usada** no corpo da procedure.
- **plpgsql** é a **linguagem procedural** do PostgreSQL, que permite:
 - Controle de fluxo: **IF, LOOP, CASE**
 - Variáveis
 - Comandos SQL internos
 - Mensagens com **RAISE NOTICE**, etc.

Sintaxe-Stored Procedures

AS \$\$... \$\$

- Delimitadores \$\$ indicam o início e o fim do **bloco de código da procedure**.
- Tudo entre \$\$ é interpretado como código em plpgsql.

Sintaxe-Stored Procedures

BEGIN ... END;

- Bloco **principal de execução** da procedure.
- Dentro dele, escreve os comandos SQL e lógicas que serão executadas quando a procedure for chamada.

Sintaxe-Stored Procedures

Dentro do bloco **BEGIN . . . END**

Pode colocar:

- Comandos INSERT, UPDATE, DELETE, SELECT
- Blocos de controle (IF, LOOP, etc.)
- Mensagens (RAISE NOTICE)
- Manipulação de erros (EXCEPTION)

Exemplo: RAISE NOTICE 'Olá, esta é uma procedure!';

Parâmetros de Entrada-Stored Procedure

```
CREATE PROCEDURE exibir_nome(nome TEXT)
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE NOTICE 'Nome informado: %', nome;
END;
$$;
```

Executando

- CALL exibir_nome('Maria');

Parâmetros de Saída-Stored Procedure

Procedures não retornam valores diretamente, mas podem usar OUT ou variáveis temporárias. Exemplo com OUT:

```
CREATE PROCEDURE somar(IN a INT, IN b INT, OUT  
resultado INT)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    resultado := a + b;  
END;  
$$;
```

Executando

- CALL somar(10, 5, result);

Exemplo 1

- **Calcular o total de uma compra com base nos produtos associados a essa compra.**

```
CREATE OR REPLACE PROCEDURE recalcula_total(p_nrocompra INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total MONEY;
BEGIN
    SELECT SUM(p.preco * t.qtde)
    INTO v_total
    FROM Tem t
    JOIN Produto p ON t.codprod = p.codprod
    WHERE t.nrocompra = p_nrocompra;

    -- Atualiza o total da compra
    UPDATE Compra
    SET total = COALESCE(v_total, 0::money)
    WHERE nrocompra = p_nrocompra;
END;
$$;
```

Interessante

COALESCE A função retorna um primeiro argumento não NULL da lista de argumentos

Exemplo 2 - Procedure com Iteração

- Cria uma procedure que
 - **Recebe um CPF de cliente.**
 - **Percorre todas as compras** desse cliente.
 - Para cada compra, **recalcula o total** com base nos produtos relacionados.
 - **Atualiza o total** em cada registro da tabela Compra.

Assim, usando **iteração com FOR** para percorrer múltiplos registros e realizar operações em lote.

Exemplo 2 - Procedure com Iteração

```
CREATE OR REPLACE PROCEDURE recalcula_totais_cliente(p_cpf CHAR(11))
LANGUAGE plpgsql
AS $$
DECLARE
    v_compra RECORD;
    v_total MONEY;
BEGIN
    -- Loop por todas as compras do cliente
    FOR v_compra IN
        SELECT nrocompra
        FROM Compra
        WHERE cpf = p_cpf
    LOOP
```

Exemplo 2 - Procedure com Iteração

```
-- Recalcula total para cada compra
SELECT SUM(p.preco * t.qtde)
  INTO v_total
  FROM Tem t
  JOIN Produto p ON t.codprod = p.codprod
  WHERE t.nrocompra = v_compra.nrocompra;

-- Se não houver produtos, define total como zero
IF v_total IS NULL THEN
  v_total := 0;
END IF;

-- Atualiza a tabela Compra
UPDATE Compra
SET total = v_total
WHERE nrocompra = v_compra.nrocompra;

-- Mensagem de log (opcional)
RAISE NOTICE 'Compra % atualizada com total = %', v_compra.nrocompra, v_total;
END LOOP;
END;
$$;
```